

Introduction to Neural Networks

An overview

Linear models

A **linear model** is a model that **assumes data is linearly separable**

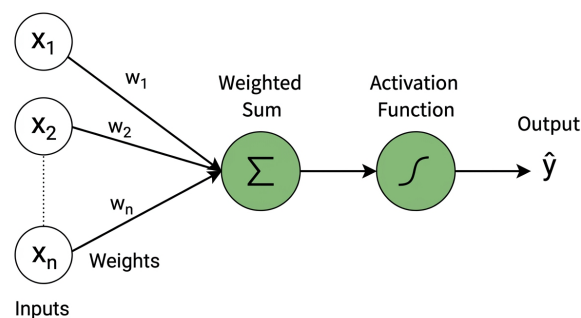
Perceptron

An example of **linear model** is the **perceptron**:

- **Iterative training** approach
 - For each example a prediction is computed as weighted sum of input plus bias
 - If the prediction is wrong, the weights and the bias are updated
- The use of the **gradient descent** makes the **correction** more **subtle**

An **artificial neuron** (= a **perceptron**):

- Is a **non-linear parameterized function**
- **Has a restricted output range**



Rosenblatt

Perceptron as a machine for linear classification

It consists of these **components**:

- **Weights** in input
 - The **bias** is an **extra weight** b with **constant input** $x_0 = 1$
- The **activation function** $h(x)$ is **stepwise function**

$$h(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

- If the result is larger than the threshold returns 1, otherwise 0

Minsky and Papert

A **single perceptron can't solve complex functions**

- As it is a linear model → **assumes data is linearly separable**
- Such as the **XOR problem**

Multi-layer Perceptron

To solve complex functions **we can combine** the use of **multiple artificial neurons**

- Densely **connected**
- Allows to realize **compositions of non-linear functions**

Each layer corresponds to **a function**

Feed-Forward Neural Networks

An **example of MLP** (= Multi-Layer Perceptron) are **Feed-Foward Neural Networks**

- The **information flows from input to output** without loops
 - Represented **as a DAG** (= Directed Acyclic Graph)
 - **Each layer implements a non-linear function**
 - **Input layer: receives** data
 - **Hidden layers: compute intermediate** representations of data
 - **Output layer: returns** the final outcome

→ The result is a combination of non-linear functions

First AI Winter

We don't know the desired **predictions of hidden layers**

- The perceptron **learning rule can't be applied**
- We **only know** the desired **target of output layer**

Backpropagation

We can apply the **idea of backpropagation** to train the MLP:

- **Forward propagation** → **input is propagated** through the net up to the output
- **Error estimation** → the **output is compared** with the **real value** using a loss function
- **Backpropagation** → the **error is propagated backward** in the net
 - To **compute gradient descents** for each layer
 - To **update all the weights** in the net

The **learning** consists of an **optimization** phase

- We aim to **adjust the weights** of the net such that the **cost function is minimized**

CNN & LSTM

MLPs evolve to:

- Convolutional Neural Networks (**CNNs**) → **apply** the **convolution** operation in at least one layer
- Recurrent Long-Short Term Memory Networks (**LSTMs**) → **networks** able to deal with **memory**

Second AI Winter

Neural Networks **weren't able to exploit many layers**

- The more layers, the higher risk of overfitting
- Problem of the vanishing gradient
 - In training several small numbers are multiplied
 - → The gradient rate used in gradient descent becomes smaller and smaller step after step
- Lack of processing power
- Lack of available data

Kernel Methods

Kernel Machines such as SVM suddenly **became popular**

- Similar accuracy to neural networks
- Have much fewer heuristics and parameters than neural networks
- Have decent proofs on generalization tasks

Learning Deep Belief Nets

New approaches to initialize the network weights:

- **Train each layer one by one** with **unsupervised learning**
- **Fine tuning the weights** with a **round of supervised learning**
- **State of the art** performance on famous datasets (such as MNIST)